

Flink as an isotope collector

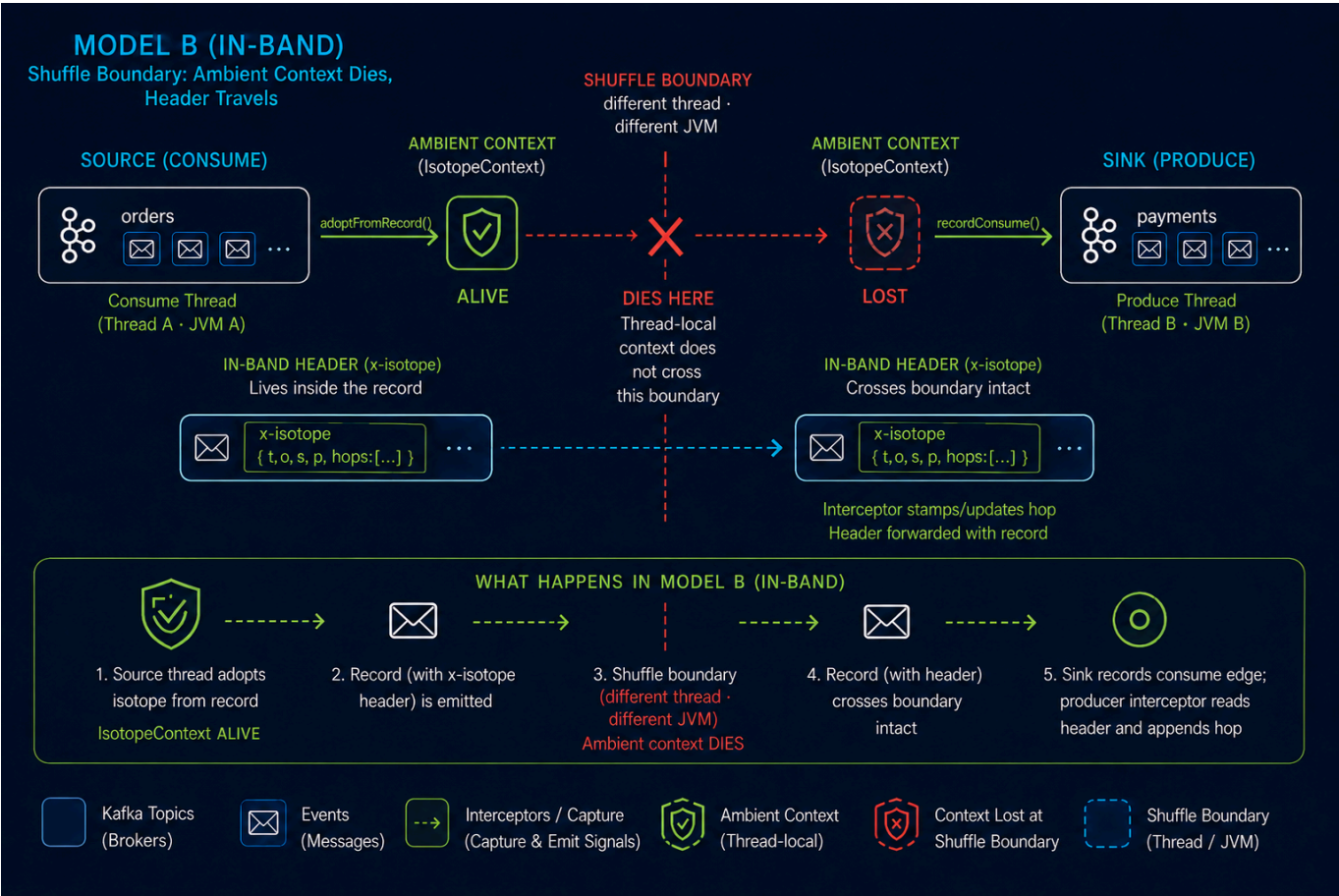
Two propagation models now coexist in this project. They emit the identical wire format, so `05_isotope_view.fql` and every report is unchanged and cannot tell the collectors apart.

	Ambient propagation	In-band propagation
Where	Services in <code>app/</code>	Flink SQL (CP + CCAF)
Carrier	<code>IsotopeContext ThreadLocal</code>	The record's own Kafka headers
Stamped by	<code>IsotopeProducerInterceptor.onSend()</code>	<code>ISOTOPE_APPEND_HOP(...)</code> in <code>INSERT INTO</code>
Activation	<code>interceptor.classes</code> config	<code>CREATE FUNCTION</code> + a writable <code>headers</code> column

Why a second model was needed

Ambient propagation is correct where it runs: one thread owns a whole consume→produce hop, so a `ThreadLocal` set at consume time is still there at `send()` time.

It cannot work in Flink. A shuffle serializes a record and resumes it on a different thread, usually in a different TaskManager JVM, so anything set *beside* the record is gone. Flink SQL is stricter still — there is no user-visible thread to attach to and no way to carry an opaque value past the planner, which is free to reorder, fuse, and eliminate rows.



The shuffle boundary is the whole argument. Ambient context lives *beside* the record and dies there; the header lives *inside* it and crosses intact.

So in-band propagation moves the isotope *into* the record — which does not make the planner gentler, it makes the isotope something the planner is **obligated to respect**. `ISOTOPE_APPEND_HOP(...)` is an ordinary scalar expression over column values, and the planner may still reorder, fuse, push down, or split that plan across a shuffle. What it may not do is change the values the query computes. Carrying a column value is precisely the thing no rewrite is allowed to optimize away, so the isotope arrives at the sink intact.

The `ThreadLocal` had no such protection, for a precise reason: no expression in the query referenced it, so the planner saw no dependency and owed it nothing. The fix is not to work around the optimizer but to state the requirement in terms it already guarantees.

Two consequences follow, both intentional. The expression references `headers`, so projection pushdown must now keep that column alive from source to sink — a real dependency, visible to the optimizer. And Flink assumes a `ScalarFunction` is deterministic unless told otherwise, meaning it may constant-fold one, eliminate it as a common subexpression, or recompute it on failover; that is why the hop timestamp is a parameter rather than a clock read inside `eval()`. See [Constraints](#).

Note this is not a Flink limitation being worked around. Nothing in Flink core needs to change; the ambient *delivery mechanism* is what doesn't port, while the isotope format and semantics port intact.

What is wired, and where

Flink collects onto a **new parallel topic**, `orders.flink_enriched`. It reads `orders.placed` and re-emits each record with one extra hop. The three-stage service pipeline is untouched — `orders.{placed,enriched,fulfilled}` still belong entirely to ambient propagation — and Flink now appears as a producer in `topology_1m` and `bipartite_topology_1m`, where it was previously invisible.

Piece	CP (Minikube)	CCAF
UDF registration	<code>01_register_functions.fql</code>	<code>register_isotope_append_hop</code> in <code>setup-confluent-flink.tf</code>
Writable sink table	<code>07_flink_collector_sink.fql</code>	<code>flink_collector_sink</code> + two <code>ALTERs</code> in <code>setup-confluent-flink.tf</code>
Collector INSERT	<code>75_flink_collector.fql</code>	<code>insert_flink_collector</code> in <code>setup-confluent-flink.tf</code>
Teardown	<code>99_teardown.fql</code>	<code>terraform destroy</code>

`IsotopeReportsJob` registers `ISOTOPE_APPEND_HOP` from the classpath and runs the collector INSERT alongside the seven report INSERTs in the same `StatementSet`.

The writable headers column

The sink declares `headers` **without** `VIRTUAL`, which is what makes it writable — virtual metadata columns are read-only and excluded from the `INSERT INTO` target schema. On CCAF the equivalent is

`ALTER TABLE ... MODIFY \headers` MAP<STRING, STRING> METADATA, since the four existing ALTERs add the column as METADATA VIRTUAL`.`

Once persisted, the column is **mandatory in every INSERT INTO** that table. Only make it writable on tables Flink actually writes.

The source tables keep `MAP<STRING, BYTES>` (the Kafka connector's native metadata type) and the call site casts to `MAP<STRING, STRING>` on the way in. That keeps one UDF signature across both runtimes instead of forking the Java, and `MAP<STRING, STRING>` is the type Confluent documents for reading and writing headers.

The hop timestamp is passed in as `UNIX_TIMESTAMP() * 1000` rather than read inside `eval()` — a built-in the planner already knows not to fold, which keeps the non-determinism somewhere the optimizer can see it rather than hidden in the Java. See [Why a second model was needed](#).

CCAF table shape

`DESCRIBE \orders.placed``` against the live cluster returns:

Column	Type	Extras
key	BYTES	BUCKET KEY
val	BYTES	
headers	MAP<STRING, BYTES>	METADATA FROM 'headers' VIRTUAL

The Topic Catalog auto-imported it as the bytes-pair table rather than deriving columns from the Protobuf schema, so the CCAF sink mirrors (`key`, `val`) while the CP side declares a single `value` BYTES column locally. Each runtime keeps the shape its own catalog produces; only the header handling is shared.

Constraints

1:1 statements only — tracing, not provenance. This is a deliberate boundary, not an unfinished feature, and the distinction is worth stating exactly.

An isotope records an *itinerary*: one identity, and the ordered hops it visited. Structurally a **path**. Provenance records a *derivation*: for each output, the inputs that produced it. Structurally a **DAG**, and inherently many-to-one wherever data combines. The two coincide as long as every step is 1:1 — a DAG whose every node has exactly one parent is still a path — which is why `IsotopeAppendHop` and `IsotopeProducerInterceptor` both work.

A windowed aggregate breaks the coincidence. A SUM over 1,000 records has 1,000 parents, and a truthful provenance record would name all of them; the isotope format has no vocabulary for more than one. So in-band propagation is restricted to 1:1 statements, where a trace *is* a valid derivation record. Nothing emits a "representative" trace ID through an aggregation, because that would not be incomplete provenance — it would be a fabrication that reads as provenance.

Note this is a property of the model, not of Flink: Kafka Streams and Spark face it identically. And in this pipeline it costs nothing, because the fan-in statements are all reports, and reports are terminal *as traced*

records. `isotope_report_*_1m` is read by dashboards, never re-consumed as a business event that must carry a hop chain onward.

The trace-RCA report is the one case that reads a report rather than the event stream — `setup-ccaf-ai.tf` joins `isotope_report_stuck_trace_1m` to `ML_PREDICT` and emits an AI-written root cause. It is not a counterexample on either count: it is 1:1 (one alert in, one analysis out, via `LATERAL TABLE`), and it carries `trace_id` forward as a *column*, not as an isotope header, so it never re-enters the hop chain. Note it is CCAF-only and gated on `var.enable_trace_rca`; CP runs seven reports, CCAF seven or eight.

The 1:1 boundary happens to land exactly where tracing needs to reach.

Full provenance would be a format change rather than a constraint to work around: the record carries a fresh trace, and a side-channel *merge-edge* topic records (`merge_trace_id`, `window`, `operator`, `contributing_traces`) — the same shape `isotope_consume_edge_markers` already uses for consume edges. In-band is not an option; 1,000 UUIDv7s is 16 KB of raw ID against a 1 MB `max.message.bytes`, and `Isotope`'s existing `MAX_HOPS` and `truncated` flag already concede the format cannot carry unbounded history along a single path. That work is only worth doing if a stateful Flink stage ever sits *mid-pipeline* — an enrichment join, a windowed dedup — whose output is still a business event someone traces further.

Header keys must be unique. Confluent's ALTER TABLE reference states multi-key headers are unsupported. Isotope uses distinct keys throughout, but this rules out ever expressing hops as repeated same-key headers.

kafka-clients linkage. `Isotope` declares

`fromHeaders(org.apache.kafka.common.header.Headers)`. `IsotopeAppendHop` never calls it, but the JVM resolves that descriptor when linking the class, so `Headers` must be present at **runtime**, not merely at compile time.

`compileOnly` was tried first, on the assumption that the Flink Kafka connector would supply `kafka-clients`. It does not on CCAF. Probed against the live compute pool, `CREATE FUNCTION` succeeded and the statement planned and reached RUNNING — then died on the first row:

```
phase: FAILED
detail: UDF invocation error: exception raised in the user function code
        Message: org.apache.kafka.common.header.Headers
```

That failure timing is the trap: CCAF's function classloader does not expose `kafka-clients`, and nothing at registration time says so.

So `ptf/build.gradle` bundles `kafka-clients` (`transitive = false`, so no compression codecs) and relocates `org.apache.kafka` → `ai.signalroom.shaded.kafka`, which rewrites `Isotope`'s own reference in step and keeps the bundled copy from ever meeting the runtime's. It then ships **only** `common/header`, dropping the rest by package: 2 classes and ~1 KB, against 4,152 classes and 12 MB for the whole client.

`minimize()` cannot do that job — a type named only in a method descriptor is invisible to bytecode reachability analysis, so `minimize` strips `Headers` and the artifact fails exactly as `compileOnly` did. Only `Isotope` is ever loaded by the UDF; `IsotopeContext`, whose descriptors do reach `ConsumerRecord` and `Producer`, is never touched, and class loading is lazy, so its missing types cost nothing.

Confirmed in production on CCAF. With the relocated build deployed, querying the collector's output returns the stamped headers:

```
SELECT `headers`['x-isotope-this-service'], `headers`['x-isotope-hop-count']  
FROM `orders.flink_enriched` LIMIT 3;
```

```
flink-enrich    2  
flink-enrich    2  
flink-enrich    2
```

`hop_count = 2` is the load-bearing part: hop 1 was the origin service producing to `orders.placed`, hop 2 is Flink. The count incremented rather than resetting, so the UDF rehydrated the inbound isotope and appended to it rather than minting a fresh trace — the same assertion `IsotopeAppendHopTest` makes, now observed end to end.

The durable fix remains a Kafka-free `isotope-format` artifact split out of `kafka-isotope`, which would make all of this unnecessary.

What this does not change

Ambient propagation is untouched. `IsotopeContext`, `IsotopeProducerInterceptor`, and the `interceptor.classes` wiring in `App.java` behave exactly as before, and remain the right model for the services.